

Программирование на Python для бизнес-аналитики
Магистратура «Бизнес-аналитика и цифровые решения в финансах» · ВШБ НИУ ВШЭ

Время, часовые пояса и метрики подписочного бизнеса

Дмитрий Тихомиров · 3 октября 2026

Проверочная работа № 2

15 минут · окончание в 13:15

Открыта только вкладка с формой

Слайды, ноутбуки, репозиторий курса и ИИ закрыты. Открытая вкладка с материалами — 0 за работу целиком.

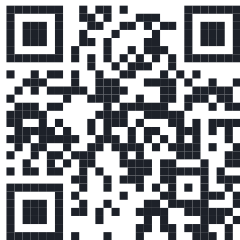
Темы 03–04 и семинары С3, С4

Сегодняшние лекции не спрашиваются

Вариант 1



Вариант 2



Вес — 2,5 % итоговой оценки. Пропустили по уважительной причине — отработка в течение двух недель, по другому варианту, не позже 17.10. Без уважительной причины — 0.

Демо-бот с первой лекции присылает алерт

Выручка от подписок за неделю с 24.08.2026 — **11,73 млн ₽**, к прошлой неделе **-18,1 %**.

Тот же код демо, что на Л1, — на нынешних данных курса.

Сколько в этом числе правды? Чтобы ответить, надо знать три вещи, которых в нём не видно.

Пояс суток

В чьём времени кончается день и начинается неделя: UTC, Москвы, Владивостока?

Границы периода

Все ли дни недели есть в данных — и сравниваем ли мы равное с равным?

Знаменатель

Когда метрику делят — на кого: на клиента, на подписку, на человека с доступом?

Четыре типа времени в pandas

Timestamp	момент: «31 января, 10:00»
Timedelta	длительность: «30 суток»
Period	отрезок календаря: «февраль 2026»
DateOffset	календарный шаг: «плюс месяц»

```
t = pd.Timestamp("2026-01-31 10:00")
print(t + pd.DateOffset(months=1))
print(t + pd.Timedelta(days=30))
print(pd.Period("2026-02", freq="M").days_in_month)
print(pd.Series([t]).dtype)
```

```
2026-02-28 10:00:00
2026-03-02 10:00:00
28
datetime64[us]
```

- «Плюс месяц» и «плюс 30 дней» приводят в разные даты: календарный месяц бывает и 28, и 31 сутки.
- В pandas 3 точность зависит от того, откуда пришло время: строки и время из базы дают микросекунды, [us], даты без времени — секунды, [s]. В старых учебниках везде [ns].

Прогон 02.10, окружение курса: pandas 3.0.5, без pyarrow.

Что приезжает из базы: время с поясом

```
pay = q("select payment_id, paid_at, amount from prime.payments "  
        "where paid_at >= '2026-07-01' and paid_at < '2026-07-02'")  
print(pay["paid_at"].dtype)  
print((pay["paid_at"] < pd.Timestamp("2026-07-01 12:00")).sum())
```

```
datetime64[us, UTC]
```

```
TypeError: Invalid comparison between dtype=datetime64[us, UTC] and Timestamp
```

```
print((pay["paid_at"] < pd.Timestamp("2026-07-01 12:00", tz="UTC")).sum())
```

```
3385
```

- В базе `paid_at` — момент времени в UTC, и pandas привозит его вместе с поясом. Это **aware**-время. Строка без пояса — **naive**-время: часы на стене, неизвестно чьи.
- Сравнивать их pandas отказывается, и это защита, а не помеха. Ту же ошибку вы видели на Л4 со справочником цен.

`q()` — функция из ноутбуков семинаров, сессия базы в UTC. Прогон 02.10, окружение курса: pandas 3.0.5, без pyarrow.

tz_localize: приписать пояс времени без пояса

Когда нужна: время пришло без пояса — из CSV, из Excel, строкой от коллеги, — а вы знаете, по чьим часам оно записано.

```
naive = pd.Timestamp("2026-04-30 20:00")
vl = naive.tz_localize("Asia/Vladivostok")
utc = naive.tz_localize("UTC")
print(vl)
print(utc)
print(utc - vl)
```

```
2026-04-30 20:00:00+10:00
2026-04-30 20:00:00+00:00
0 days 10:00:00
```

- Цифры на часах не меняются — появляется пояс. Только теперь это момент времени.
- Какой пояс приписать, pandas не знает: это ваше знание об источнике. Те же 20:00 по Владивостоку и по UTC — два момента, между ними 10 часов.
- Риск: приписали не тот пояс — всё сдвинулось на часы, и ни одной ошибки.

Прогон 03.10, окружение курса: pandas 3.0.5, без pyarrow.

tz_convert: ТОТ ЖЕ МОМЕНТ В ДРУГОМ ПОЯСЕ

Когда нужна: пояс у времени уже есть — из базы пришёл UTC, — а сутки или часы нужны другие: отчёт по Москве, местное время клиента.

```
t = pd.Timestamp("2026-04-30 20:00", tz="UTC")
msk = t.tz_convert("Europe/Moscow")
print(msk)
print(t.tz_convert("Asia/Vladivostok"))
print(msk == t)
print(msk.tz_localize(None))
```

```
2026-04-30 23:00:00+03:00
2026-05-01 06:00:00+10:00
True
2026-04-30 23:00:00
```

- Цифры на часах меняются, а момент тот же: сравнение с t даёт True.
- Дата тоже может смениться: в Москве ещё 30 апреля, во Владивостоке уже 1 мая.
- tz_localize(None) снимает пояс — так время готовят к выгрузке в Excel.

Как выбрать: пояса нет — tz_localize, один раз, при загрузке. Пояс есть — tz_convert, сколько угодно раз. Перепутаете — pandas упадёт и сам назовёт нужную функцию.

Прогон 03.10, окружение курса: pandas 3.0.5, без pyarrow.

Клиенты ПРАЙМ живут в 11 поясах

Пояс	UTC	Клиентов
Europe/Kaliningrad	+2	4 894
Europe/Moscow	+3	274 856
Europe/Samara	+4	30 151
Asia/Yekaterinburg	+5	55 091
Asia/Omsk	+6	25 046
Asia/Krasnoyarsk	+7	34 963
Asia/Irkutsk	+8	25 063
Asia/Yakutsk	+9	15 202
Asia/Vladivostok	+10	19 843
Asia/Magadan	+11	9 893
Asia/Kamchatka	+12	4 998

- Все отметки времени в базе хранятся в UTC. Пояс — свойство клиента, колонка `clients.time_zone`.
- Больше половины клиентов живут в московском поясе, а разброс идёт от UTC+2 до UTC+12.
- Когда в Москве 18:00, на Камчатке уже 3 часа ночи следующего дня.

Клиенты по `time_zone`; смещение — `pd.Timestamp("2026-07-01", tz=tz).utcoffset()`. Прогон 02.10.

Местная дата платежа: у каждого клиента свой пояс

pay — успешные платежи июля 2026 с поясом владельца подписки.

```
parts = []
for tz, g in pay.groupby("time_zone"):
    parts.append(g["paid_at"].dt.tz_convert(tz).dt.date)
pay["local_date"] = pd.concat(parts)
print((pay["local_date"] != pay["paid_at"].dt.date).mean())
```

```
0.19313207933985957
```

- `tz_convert` переводит всю колонку в один пояс. Когда пояс у каждой строки свой, переводят по группам: `groupby` отдаёт пары «пояс — его строки», `concat` собирает куски обратно по индексу.
- В базе то же самое — одна строка: `(p.paid_at at time zone c.time_zone)::date`.
- В июле на другую дату в местном времени клиента приходится 19,3 % платежей, по всей базе — **19,4 %**: каждый пятый платёж.

Доля по всей базе — SQL по 3 005 335 успешным платежам. Прогон 02.10, окружение курса: pandas 3.0.5, без pyarrow.

Дыра 14–15 апреля в местном времени клиентов

Дата	По UTC, ₽	По местному, ₽
12.04	1 937 448	1 944 742
13.04	1 908 316	1 889 617
14.04	0	389 694
15.04	0	0
16.04	1 922 521	1 552 106
17.04	1 951 775	1 936 713

- По UTC пропали два дня целиком: данные выгружали по суткам UTC.
- В местном времени клиентов пропал **один** день, а два соседних — частично: 14-го есть пятая часть обычного дня, 16-го — четыре пятых.
- Дыру ищут в том поясе, в котором грузили данные. В местном времени проверка полноты найдёт не то, что случилось.

Успешные платежи всех клиентов; местная дата — по поясу владельца подписки. Прогон 02.10.

Какие сутки правильные — зависит от вопроса

Вопрос	Чьи сутки	Почему
Полна ли выгрузка за 14 апреля?	UTC	так данные грузили и хранят
Выручка в отчёте руководству	пояс отчёта, например Москва	договорённость — её пишут в подписи
Сколько клиентов оформили подписку в день регистрации?	местные сутки клиента	поведение человека живёт по его часам

- У `spend_date` в `marketing_spend` тип `date`, а у даты пояса нет. Чья это дата, знает владелец данных — в ПРАЙМ это дата по UTC.

Пояс суток — часть определения метрики. В курсе считаем в UTC (`set time zone 'UTC'`) и пишем это в подписи к числу.

Пояс — это имя, а не «+3»

```
moscow = "Europe/Moscow"  
print(pd.Timestamp("2014-01-15 12:00", tz=moscow).utcoffset())  
print(pd.Timestamp("2025-01-15 12:00", tz=moscow).utcoffset())  
pd.Timestamp("2026-03-29 02:30").tz_localize("Europe/Berlin")
```

4:00:00

3:00:00

ValueError: 2026-03-29 02:30:00 is a nonexistent time due to daylight savings time.
Try using the 'nonexistent' argument.

Смещение пояса меняют законом: Москва жила в UTC+4 с марта 2011 по октябрь 2014 года. Python это помнит — если писать имя пояса, а не прибавлять три часа.

Ошибка перенесена на две строки, текст дословный. Прогон 02:10, окружение курса: pandas 3.0.5, без pyarrow; пояса — zoneinfo, в pandas 3 по умолчанию.

В Европе часы переводят дважды в год: 29 марта 2026 года в Берлине времени 02:30 нет. В ПРАЙМ все 11 поясов российские, и часы в них не переводят.

Периоды

неполная неделя, короткий месяц, дыра в ряду

resample: агрегат по периодам календаря

resample режет ось времени на периоды и в каждом считает агрегат.

```
days = pd.to_datetime([
    "2026-01-30", "2026-01-31",
    "2026-02-02", "2026-02-03"])
x = pd.Series([10, 20, 30, 40], index=days)
print(x.resample("ME").sum())
print(x.resample("D").sum())
```

```
2026-01-31    30
2026-02-28    70
Freq: ME, dtype: int64
2026-01-30    10
2026-01-31    20
2026-02-01     0
2026-02-02    30
2026-02-03    40
Freq: D, dtype: int64
```

- Время должно стоять в индексе ряда: resample смотрит на индекс.
- В скобках — частота: "D" — день, "W" — неделя, "ME" — месяц. Месяц подписан последним днём: ME — month end.
- Агрегат пишут как после groupby. Но, в отличие от него, пустой период не пропадает: первого февраля в данных нет — в дневном ряду оно стоит строкой с нулём.

Прогон 03.10, окружение курса: pandas 3.0.5, без pyarrow.

resample на выручке ПРАЙМ — и "ME" вместо "M"

daily — успешные платежи по дням UTC: day, n, revenue.

```
s = daily.set_index(pd.to_datetime(daily["day"]))["revenue"]  
s.resample("M").sum()
```

```
ValueError: Invalid frequency: M. Failed to parse with error message:  
ValueError("'M' is no longer supported for offsets. Please use 'ME' instead.")
```

```
print(s.resample("ME").sum()["2026-05":])
```

```
day  
2026-05-31    60763772.0  
2026-06-30    60156153.0  
2026-07-31    63398990.0  
2026-08-31    59312524.0  
Freq: ME, Name: revenue, dtype: float64
```

- Первая строка готовит ряд s: to_datetime делает из колонки day время, set_index ставит её индексом, ["revenue"] оставляет одну колонку.
- resample("ME").sum() складывает выручку дней каждого месяца.
- В pandas 3 месяц пишут "ME", а час — "h". Старые "M" и "H" падают, а ими написаны учебники и ответы ИИ.

Ошибка в выводе перенесена на две строки, текст дословный. Прогон 02.10, окружение курса: pandas 3.0.5, без pyarrow.

Алерт сравнил семь дней с шестью с половиной

Что видит алерт: недели

Неделя с	Платежи, ₽	Транзакций
03.08	14 531 675	95 000
10.08	14 583 971	96 729
17.08	14 334 771	96 805
24.08	11 734 084 (-18,1 %)	83 336 (-13,9 %)

Что внутри последней недели: дни

День	Списаний	Транзакций
27.08	5 588	13 779
28.08	5 685	13 844
29.08	5 545	13 705
30.08	2 796	нет строк

- Выгрузка обрывается посреди последней недели: за 30 августа есть только половина списаний, а транзакций за этот день нет совсем.
- Алерт сравнил полную неделю с неполной. Часть «падения» на 18,1 % — не событие в бизнесе, а дни, которых в данных нет.

Перед сравнением периодов смотрят на последний день выгрузки: целый ли он.

Неделя — с понедельника, сутки UTC. Списания — успешные платежи. Прогон 02.10.

Равные окна: настоящее падение — 13 %

В обеих неделях берём понедельник–субботу: эти дни выгружены целиком.

```
a = s["2026-08-17":"2026-08-22"].sum()  
b = s["2026-08-24":"2026-08-29"].sum()  
print(round(b / a - 1, 3))
```

-0.13

- `s` — выручка по дням, в индексе даты. Срез `s["дата": "дата"]` берёт строки от даты до даты, и обе границы входят.
- `a` — выручка шести дней прошлой недели, `b` — тех же шести дней последней.

Что с чем сравнили	Изменение
Неделя к неделе: 7 дней и 6,5 дня	-18,1 %
Пн–сб к пн–сб: 6 дней и 6 дней	-13,0 %

Из -18,1 % настоящие — -13,0 %.
Остальное дал недогруженный день. За счёт чего выручка упала на 13 % — разбираем на Лб.

Неделя — с понедельника, сутки UTC. Прогон 02.10, окружение курса: pandas 3.0.5, без pyarrow.

Месяц — не единица измерения

cnt — число успешных платежей по дням.

```
n = cnt.resample("ME").sum()["2026-01":"2026-02"]
print(n)
print(n.index.days_in_month)
print((n / n.index.days_in_month).round())
```

```
day
2026-01-31    170665
2026-02-28    158265
Freq: ME, Name: n, dtype: int64
Index([31, 28], dtype='int32', name='day')
day
2026-01-31     5505.0
2026-02-28     5652.0
Freq: ME, dtype: float64
```

- Первая строка складывает платежи по месяцам. `n.index.days_in_month` — число дней в каждом месяце: 31 и 28. Последняя строка делит платежи месяца на его дни.
- В феврале платежей на 7,3 % меньше, а в пересчёте на день — на 2,7 % больше: подписка списывается раз в 30 дней, и часть февральских списаний уехала в следующий месяц.
- Месяцы разной длины сравнивают в пересчёте на день.

Месяцы по UTC. История — с 6.01.2025 по 30.08.2026: крайние месяцы неполные. Прогон 03.10.

rolling и shift: ОКНО И СДВИГ

```
x = pd.Series([10, 20, 30, 40, 50])
print(pd.DataFrame({
    "x": x,
    "roll3": x.rolling(3).mean(),
    "prev": x.shift(1),
    "growth": (x / x.shift(1) - 1).round(2)}))
```

	x	roll3	prev	growth
0	10	NaN	NaN	NaN
1	20	NaN	10.0	1.00
2	30	20.0	20.0	0.50
3	40	30.0	30.0	0.33
4	50	40.0	40.0	0.25

- `rolling(3)` идёт по ряду окном в три строки и считает агрегат окна. В строке 2: $(10 + 20 + 30) / 3 = 20$.
- Пока окно не набралось, результата нет — NaN.
- `shift(1)` сдвигает ряд на строку вниз: рядом с сегодняшним значением — вчерашнее. Так считают прирост.

Зачем: скользящее среднее сглаживает скачки между днями — виден тренд, а не шум.
«Среднее за 7 дней» и «к прошлой неделе» — обычные строки мониторинга.

Прогон 03.10, окружение курса: pandas 3.0.5, без pyarrow.

rolling(7) — это семь строк, а не семь дней

В дневном ряду `s` нет строк за 14 и 15 апреля — это дыра в данных.

```
print(s.index.to_series().diff().value_counts())
out = pd.DataFrame({
    "rows7": s.rolling(7).mean(),
    "days7": s.rolling("7D").mean(),
    "zeros7": s.asfreq("D", fill_value=0).rolling(7).mean()})
print(out.loc["2026-04-16":"2026-04-18"].round())
```

```
day
1 days      598
3 days        1
Name: count, dtype: int64
```

	rows7	days7	zeros7
day			
2026-04-16	1927853.0	1929743.0	1378388.0
2026-04-17	1930190.0	1938039.0	1384314.0
2026-04-18	1931325.0	1927768.0	1376977.0

- `rolling(7)` берёт семь строк и молча захватывает девять дней. `rolling("7D")` берёт семь дней.
- `asfreq("D", fill_value=0)` делает дыру видимой: пропавшие дни становятся строками с нулём.

С `shift` то же самое: `shift(7)` сдвигает на семь строк, `shift(7, freq="D")` — на семь дней.
Прогон 02.10, окружение курса: pandas 3.0.5, без pyarrow.

Метрики подписки

окно, знаменатель и начало периода

Воронка: от регистрации до второй оплаты



Проценты — от предыдущего шага. Ширина столбика — число клиентов.

- Клиенты, зарегистрированные в феврале–июне 2025 года: пройти все шаги у них было больше года.
- Больше всего теряется на первом шаге: подписку оформляют 70 %. Дальше почти все платят и продлевают.
- Из 54 211 подписок 9 291 (17,1 %) начались с триала.

Вторая оплата — это продление. Оплата засчитывается, пока подписка жива. Прогон 02.10.

У воронки есть окно

Подписку оформили — из 77 644

Окно	Клиентов	%
За 30 дней после регистрации	26 663	34,3
За всё время наблюдения	54 211	69,8

- Окно — срок, за который шаг должен случиться. «Конверсия 34 %» и «конверсия 70 %» — одна воронка с разными окнами: без окна число не воспроизвести.

% — от регистраций. Прогон 02.10.

Подписка за 30 дней, по месяцу регистрации

Месяц регистрации	%
май 2026	34,4
июнь 2026	34,9
июль 2026	33,6
август 2026	19,1

У зарегистрированных в августе 30 дней ещё не прошли: окно не закрылось. Воронку считают по тем, у кого было время пройти все шаги.

Одна выручка — четыре ARPU

ARPU, average revenue per user, — средняя выручка на пользователя за период: выручку делят на число пользователей. **ARPPU** делит только на платящих.

Выручка июля 2026 — **63 398 990 ₽**: успешные платежи, месяц по UTC.

Делим на	Сколько	ARPU, ₽
Зарегистрированных клиентов	485 136	130,68
Людей с доступом: владельцев и участников	331 767	191,09
Активные подписки	199 748	317,39
Подписки с успешным платежом — это ARPPU	189 157	335,17

- Кто такой «пользователь», решаете вы: клиент, человек с доступом или подписка. Между крайними ответами **2,6 раза**, и «ARPU 317 ₽» без знаменателя не воспроизводится.
- Людей с доступом считают объединением владельцев и участников — как строку ЛЮДИ в протоколе сборки.

Активна в июле: `started_at` до 01.08, `ended_at` пуст или не раньше 01.07. Прогон 02.10.

Отток и удержание: кого считаем

Отток, churn rate, — доля подписок, которые были активны на начало периода и закончились до его конца. **Удержание**, retention rate, — доля оставшихся: $1 - \text{отток}$.



Активны на 1 июля	188 410
закончились в июле	7 392

Отток за июль	3,92 %
Удержание	96,08 %

Оформлены в июле	11 338
------------------	--------

- Знаменатель — подписки, активные на начало периода. Оформленные в июле в него не входят: иначе отток зависел бы от того, сколько продали в этом месяце.

Расчёт — на следующем слайде. Прогон 02.10.

Отток за июль: расчёт

```
start = pd.Timestamp("2026-07-01", tz="UTC")
end = pd.Timestamp("2026-08-01", tz="UTC")
active = subs[(subs["started_at"] < start)
               & (subs["ended_at"].isna() | (subs["ended_at"] >= start))]
churned = (active["ended_at"] < end).sum()
churn = churned / len(active)
print(len(active), churned, f"{churn:.2%}")
print(f"{{(1 - churn) ** 12:.1%}}  {{12 * churn:.1%}}")
```

```
188410 7392 3.92%
61.9%  47.1%
```

`active` — подписки, которые начались до 1 июля и на 1 июля ещё не закончились:
`ended_at` пуст или не раньше 1 июля. `churned` — сколько из них закончилось до 1 августа.

Когортные таблицы удержания — в курсе «SQL и базы данных». Здесь — отток и удержание за период.

`subs` — все подписки: `started_at`, `ended_at`. Прогон 02.10, окружение курса: pandas 3.0.5, без pyarrow.

Последняя строка — отток за год. При том же оттоке останется 61,9 % подписок: доли перемножаются. Годовой отток — 38,1 %, а не « $12 \times 3,92 \% = 47,1 \%$ ».

LTV: прогноз по формуле и нижняя граница по данным

LTV, lifetime value, — выручка, которую подписка приносит за всю жизнь.

$$\text{LTV} \approx \frac{\text{ARPPU}}{\text{отток}} = \frac{335,17}{0,03923} \approx \mathbf{8\,543\, \text{Р}}$$

При оттоке 3,92 % в месяц подписка живёт в среднем 25,5 месяца и платит 335 Р в месяц. Это прогноз: формула считает отток и платёж постоянными.

Для решения нужна маржа, а не выручка: минус кэшбэк и выплаты партнёрам.

ARPPU и отток — с предыдущих слайдов, оба за июль 2026. Выручка «к N-му месяцу» — накопленная сумма, cumsum по месяцам жизни (30 дней). Прогон 02.10.

Подписки, оформленные в феврале 2025

Подписок	10 351
Ещё живы на 30.08.2026	4 273 (41 %)
Выручка на подписку за 1-й месяц жизни	220 Р
к 6-му месяцу	1 430 Р
к 12-му	2 585 Р
к 18-му	3 607 Р

41 % подписок ещё живы, и их выручка будет расти: по данным LTV — нижняя граница.

САС: сколько стоил клиент — и кто такой «клиент»

САС, customer acquisition cost, — стоимость привлечения одного клиента: расходы на привлечение за период делят на число клиентов, привлечённых за тот же период.

Расходы — **205 478 441 Р** за 601 день, с 6 января 2025 года по 29 августа 2026-го. Кого считать привлечённым — решение: от него САС меняется в полтора раза.

Клиенты, зарегистрированные за те же дни	Сколько	САС, Р
Все зарегистрированные	310 700	661
Оформили подписку	201 461	1 020
Хотя бы раз заплатили	198 951	1 033

- У 6 259 из этих клиентов канала нет. При разбивке по каналам они выпадают из знаменателя, а расходы остаются: САС каналов завышен в среднем на 2 %.
- `spend_date` — дата без пояса, а регистрация — момент в UTC: сутки должны совпасть.

Какой канал окупается — LTV и САС по каналам — считаете на семинаре.

`marketing_spend` и `clients.registered_at` за те же 601 день. Прогон 02.10.

Что унести с этой пары

- **Пояс суток — часть определения метрики.** В базе всё в UTC. В пояс вопроса переводят по имени пояса, а не прибавляют три часа.
- **naïve и aware не сравниваются.** `tz_convert` переводит стрелки, `tz_localize` приписывает пояс.
- **Первый и последний день выгрузки — под подозрением.** Сравнивают равные окна, месяцы — в пересчёте на день.
- **rolling(7) — семь строк, rolling("7D") — семь дней.** В ряду с дырой это разные числа.
- **У воронки есть окно, у ARPU — знаменатель, у оттока — начало периода, у LTV — горизонт.** Без них число не воспроизводится.

После перерыва — Л6: **за счёт чего** упала выручка в конце августа — и как показать это заказчику.